

Comparison of Time Series Augmentation Methods for Self-Supervised Contrastive Learning Applied to Remote Sensing Time Series

Master Thesis

Report submitted in partial fulfillment of the requirements for the Master's degree in Data Science and Artificial Intelligence of the French-Azerbaijani University (UFAZ)

Student name: Arzuman Hasanov

Specialty: MSc in Data Science and Artificial Intelligence

Internship period: 03-02-2025 to 20-06-2025

Supervisors: Prof. Baptiste Lafabrègue, Prof. Pierre Gançarski, Phd. Antoine Saget

Submission date: 28-05-2025

Abstract

This work investigates the performance of different time series augmentation techniques under Self-supervised learning (SSL) frameworks applied to remote sensing time series. The majority of satellite-2 data is unlabeled, and manual labeling is time-consuming and highly costly. To address this problem, SSL aims to learn meaningful features even with small labeled datasets. In this approach, we rely on augmentation techniques. Data augmentations are well adapted to images, but applying them to remote sensing time series is still a challenge. In this study, we evaluated 10 individual augmentation methods including Resampling, Window Slice, Resizing, Jittering, Time Masking, Mixup, Window Warping, Temporal Dropout, Scaling, StiefelGen, and 6 combinations of the best-performing ones among them, across four popular frameworks: SimCLR, VICReg, MoCo, and BYOL, using the FranceCrops dataset. The results show that combining augmentation methods, especially Resampling, Temporal Dropout, and Window Slice perform better than using a single method. They reduce label dependence approximately 2-5 times. The combination Resampling + Window Slice is the best-performed combination among all the methods we implemented. The frameworks VICReg and BYOL benefited the most improvements from this strategy, SimCLR also showed improvements, but MoCo showed less satisfied results in general. The findings also suggest that the augmentations that alter the temporal structure of the data are more effective for remote sensing time series data. These strategies offer an effective way of learning feature representations without depending on large labeled datasets.

Keywords: Combined Augmentations, Contrastive Learning, Time Series, FranceCrops, Remote Sensing, Self-Supervised Learning.

Table of Contents

1	Introduction	3
2	Literature Review	4
2.1	Self-supervised learning in Remote Sensing	4
2.2	Contrastive learning frameworks	5
2.2.1	Simple Framework for Contrastive Learning of Visual Representations(SimCLR)	5
2.2.2	Variance-Invariance-Covariance Regularization (VICReg)	5
2.2.3	Momentum Contrast (MoCo)	6
2.2.4	Bootstrap Your Own Latent (BYOL)	7
2.3	Data augmentation for Time Series	7
3	Methodology	8
3.1	Data Augmentation Methods	8
3.1.1	Resampling	8
3.1.2	Window Slice	9
3.1.3	Time Masking	10
3.1.4	MixUp	11
3.1.5	Jittering	12
3.1.6	Resizing	13
3.1.7	Window Warping	13
3.1.8	Scaling	14
3.1.9	Temporal Dropout	15
3.1.10	StiefelGen	16
3.2	Experimental Setup	17
3.2.1	Dataset	17
3.2.2	Architecture	18
3.2.3	Training and Evaluation Settings	19
4	Experimental Results	19
5	Discussion	22
5.1	Findings	22
5.2	Limitations	22
6	Conclusions	22

1 Introduction

The Sentinel-2 satellite constellation records multispectral images of the Earth (Drusch et al., 2012; Gascon et al., 2017). However, this huge amount of satellite image time series (SITS) is unlabeled. Labeling manually is highly costly. As a result, only a small volume of this data is used.

To handle this limitation, self-supervised learning (SSL) is used. SSL provides effective representation learning even with a small labeled dataset (Hénaff et al., 2020).

Masked based SSL methods are based on masking and regenerating masked parts of the input. While masking methods are encouraging, applying them to SITS is challenging, because of its characteristics.

Among SSL methods, contrastive learning proved its effectiveness in many fields, especially with models like MoCo (He, Fan, Wu, Xie, & Girshick, 2020), and SimCLR (T. Chen, Kornblith, Norouzi, & Hinton, 2020). These models are mainly based on the data augmentation process for creating positive and negative pairs, which were originally designed for images (T. Chen, Kornblith, Swersky, Norouzi, & Hinton, 2020), but their application to time series is still challenge (Liu, Alavi, Li, & Zhang, 2024).

We aim to address these challenges by studying different augmentation techniques designed for multispectral time series. A detailed comparison of the following transformations and 6 combinations of well-performed transformations among them are applied to the FranceCrops dataset: Resampling, Window Slice, Resizing, Jittering, Time Masking, Mixup, Window Warping, Temporal Dropout, Scaling, and StiefelGen.

The goal is to analyze the impact of transformations and identify effective ones for learning feature representations from unlabeled data. The linear evaluation is performed using the following SSL frameworks: SimCLR, VICReg, MoCo, and BYOL, for different numbers of training samples (5, 10, 20, 50, 100) per class.

We hypothesize the following outcomes based on the combined augmentation techniques and frameworks:

- Combining different augmentation techniques, particularly Temporal Dropout, Resampling, and Window Slice, will outperform individual techniques and reduce label dependence.
- BYOL and VICReg are expected to show better performance with combined techniques than SimCLR and MoCo, as they don't rely on negative samples and are more sensitive to feature diversity.

The report has six main sections. Section 2 covers related works, SSL in remote sensing, key frameworks (see Sections 2.2.1–2.2.4), and time series data augmentation (see Section 3.1). Section 3 is containing the methodology part with a description of the augmentations we used (see Sections 3.1.1–3.1.10), then the experimental setup with dataset, architecture, and how we trained the models (see Sections 3.2.1–3.2.3). The next section (Section 4) shows our results. In Section 5, we talk about the findings and limitations. In the end, Section 6 refers conclusions part.

2 Literature Review

This section presents SSL in remote sensing, contrastive learning frameworks, and augmentation techniques designed for time series.

2.1 Self-supervised learning in Remote Sensing

The growing interest in general-purpose models for Earth observation has made it necessary to develop Remote Sensing Foundation Models (RSFMs) trained on diverse large-scale satellite data to learn meaningful representations. Within SSL, the following categories have become important: masked modeling and contrastive learning.

Recent approaches have improved masked modeling methods to more effectively handle time series data. Traditionally, models based on Masked Autoencoders (MAE) (He et al., 2022) mask a part of data and try to reconstruct that part. To adapt this idea to time series, recent models like SatMAE (Cong et al., 2022) and Prithvi (Jakubik et al., 2023) implement masking across temporal and spectral channels too, which makes models better perform with time series data. ScaleMAE (Reed et al., 2023) handles differences in scale by reconstructing different frequencies one by one. These transformer-based approaches face scalability challenges, because attention becomes much more complex as data grows. To handle this problem, they often use fewer time steps.

To handle temporal scalability, Presto (Tseng, Zvonkov, Purohit, Rolnick, & Kerner, 2023) uses a different strategy, it ignores spatial information, and treats each pixel as an independent multivariate time series. This lets Presto work with extended time sequences (for example, 10 months of data), and also combine data from multiple data sources. Using this approach Presto shows that effective performance is still achievable without spatial dependencies.

In contrastive learning, it makes instance similarities optimal for better representation learning. An early remote sensing example is Tile2Vec (Jean et al., 2019), which uses triplets of image tiles (an anchor, a positive neighboring tile, and a negative distant tile) to apply triplets loss and to get spatial representations in satellite images. This is base for later contrastive approaches.

Extending this idea, Seasonal Contrast (SeCo) (Mañas, Lacoste, Giró-i Nieto, Vazquez, & Rodríguez, 2021) uses a shared encoder with three distinct projection heads. The first one learns invariance to image augmentations, the second one learns invariance to changes impacted by the season (images at different times), and the third one combines these invariances. But, the model does not handle time series data directly.

Other models adapt this study to multi-modal data. For instance, some methods study the contrastive learning frameworks, such as DINO, MoCo, and masked modeling (Y. Wang et al., 2023).

The more recent methods are focused on improving model generalization for diverse sensors. One of the examples is SkySense (Guo et al., 2024), based on a multi-level contrastive learning strategy, which produces robust representations of objects, images, etc. It uses a high-resolution WorldView-3/4 dataset. However, SkySense does not use temporal data augmentations, which is the core of contrastive learning. The limitation is that SkySense requires high computational costs, so it makes it unsuitable to use.

2.2 Contrastive learning frameworks

Contrastive learning tries to minimize the distance between the representations of similar samples, known as positive pairs, in the latent space. Usually, different augmentation methods are used to create these positive pairs from the same sample. But if we concentrate only on the similarity of positive pairs, representation collapse can happen, where all samples are mapped into the same representation by the model.

2.2.1 Simple Framework for Contrastive Learning of Visual Representations(SimCLR)

SimCLR framework is originally developed for images, but adapted to time series (T. Chen, Kornblith, Norouzi, & Hinton, 2020). The idea is creating two augmented views (x_i and x_j) of each input sample to form a positive pair using random transformations (e.g., jittering, resizing, window slice), as illustrated in Figure 1.

We pass x_i and x_j to an encoder $f(\cdot)$, then to a projection head $g(\cdot)$, and get latent vectors z_i and z_j . Similar representations are taken as positive, and dissimilar representations are negative pairs, this is the rule of contrastive learning.

SimCLR doesn't need a memory bank. It relies on large batch sizes.

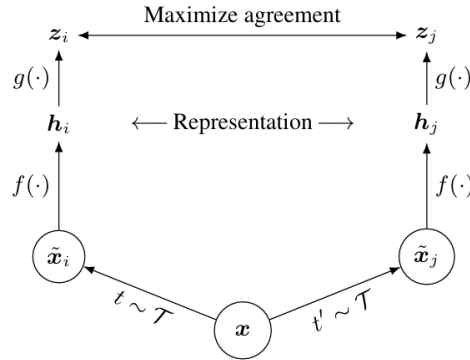


Figure 1: SimCLR architecture, from T. Chen, Kornblith, Norouzi, and Hinton (2020)

We don't need the projection head after training, but we still use the encoder.

2.2.2 Variance-Invariance-Covariance Regularization (VICReg)

Vicreg framework learns features by increasing the consistency of embedding vectors for different views of the same sample (Bardes, Ponce, & Lecun, 2022). It effectively overcomes representation collapse which encoders provide useless vectors by using the loss function of variance, invariance, and covariance.

- **Invariance** makes representations similar by using mean square error (MSE), which means that even if they look different, they are still the same sample.
- **Variance**: Variance makes sure that not all embeddings are the same, it penalizes low variance. Without this, the model may always learn the same features, which is useless.

- **Covariance** makes sure that the features in the representation vector are unique, without redundancy. It applies penalizing where different embedding dimensions are strongly correlated.

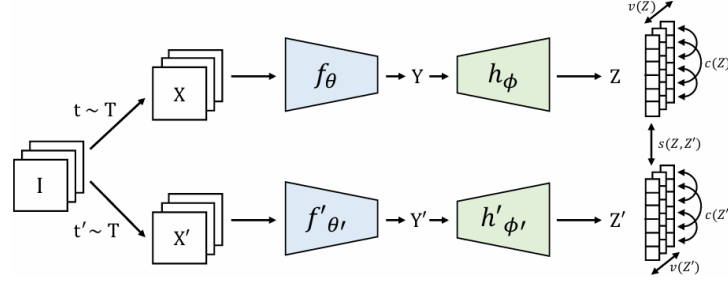


Figure 2: VICReg architecture, from Bardes et al. (2022).

Figure 2 illustrates the VICReg framework. For a given input sample, it generates two augmented views (X_1 and X_2) using augmentation methods (Scalin, Window Slice, etc). Then we pass these views to the encoders (f and f') individually to get representations (Y and Y'). Then we map representations into the embeddings Z and Z' . After that step, three losses are calculated: variance, invariance, and covariance. Unlike other frameworks, VICReg doesn't need batch normalization, weight sharing, memory bank, etc.

2.2.3 Momentum Contrast (MoCo)

MoCo framework (He et al., 2020) is originally designed for images, but adapted to time series. We implement augmentations to input sample and get 2 different views of the same sample. The idea is making dynamic and consistent dictionary of queue and encoder (momentum updated).

MoCo uses separated dictionary size, it is good at negative sampling even with small batch sizes, where the gpu has limited capacity.

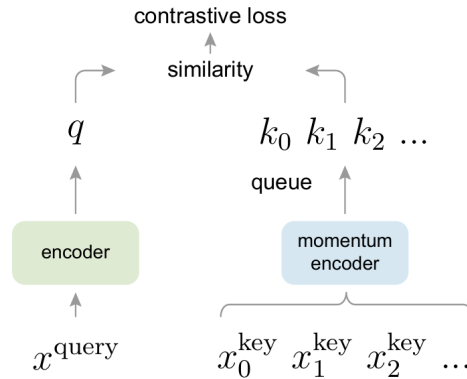


Figure 3: MoCo architecture, from He et al. (2020)

Figure 3 illustrates the MoCo framework. It implements augmentations to the same input sample and gets two different views, then passes them to a query and momentum encoders separately.

The next step is the comparison of the query and a queue of keys. Using contrastive loss, it marks similar features as positive pairs and dissimilar ones as negative pairs. The query encoder is also used for downstream tasks to extract features.

2.2.4 Bootstrap Your Own Latent (BYOL)

The **BYOL** framework uses two networks: target and online, without depending on negative pairing (Grill et al., 2020).

Given an input sample from the Francencrops dataset, we generate two different views of the same input using augmentation techniques like Window Slice, Resampling, Temporal Dropout, etc.

The encoders provide the representations, then the projector reduces dimensionality. The view of the online network is passed to the prediction head to produce prediction as illustrated in Figure 4. The training objective tries to reduce negative cosine similarity of the online predictions and the target projections as much as possible.

This approach makes the model more resistant to representation collapse, and still learn representations without negative sampling.

After training the components are removed except encoder, which is using in downstream tasks.

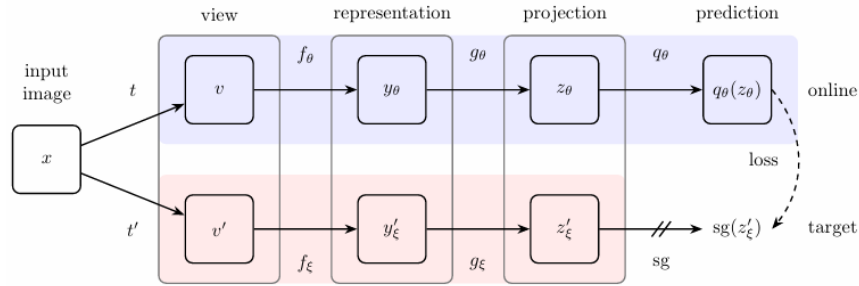


Figure 4: BYOL architecture, from Grill et al. (2020).

2.3 Data augmentation for Time Series

In contrast to masked modeling, contrastive methods typically implement spatial augmentations (Rotation, Cropping, etc) that are not directly implementable to time series data. It has a diverse nature, which makes it difficult to augment.

Reference Iwana and Uchida (2021) investigates 12 augmentation techniques: Jittering, Permutation, Flipping, Magnitude Warping, Time Warping, Scaling, Window Warping, SPAWNER, Weighted Dynamic Time Warping Barycentric Averaging, Slicing, Random Guided Warping, and Discriminative Guided Warping. These techniques were evaluated using various datasets to show the strengths and limitations of each method.

While Iwana and Uchida (2021) presents a detailed study and evaluation of various augmentation techniques for the general time series classification, their work does not specifically analyze remote sensing data. In contrast, in this work, we concentrate on evaluating augmentations designed for SITS specifically and consider the unique characteristics of the signals of Sentinel-2.

We compare the following transformations: Resampling, Window Slice, Resizing, Jittering, Time Masking, Mixup, Window Warping, Temporal Dropout, Scaling, and StiefelGen and 6 combinations of well-performed transformations among them.

3 Methodology

This section presents the augmentation methods we implemented and the experimental setup.

3.1 Data Augmentation Methods

Data augmentation is a key component of SSL, it is not only useful for expanding the training data, but also it increases model robustness against overfitting. Unlike image data, time series have temporal structures, which makes effective augmentations more complex. This section contains several augmentation techniques designed for satellite time series. Each subsection explains a specific method, its objective, and its application.

3.1.1 Resampling

This section is adapted from the Github repository¹, which developed by our team.

Given a time series $\mathbf{S} = \{s_1, \dots, s_T\} \in \mathbb{R}^{T \times C}$, with T time steps and C channels, resampling carried out in three steps.

Step 1: As illustrated in Fig. 5b, we upsampled the time series to $T_{\text{up}} = 2 \times T$ by using linear interpolation:

$$\mathbf{S}_{\text{up}} = f_{\text{linear}}(\mathbf{S}) \in \mathbb{R}^{T_{\text{up}} \times C} \quad (1)$$

Step 2: The next step is sampling two subsequences $\mathbf{S}_{\text{sub}}^1$ and $\mathbf{S}_{\text{sub}}^2$, each having $T/2$ time steps, as shown in Figure 5c. There are two restrictions:

- The time steps must be unique: $\mathcal{X}_{\text{sub}}^1 \cap \mathcal{X}_{\text{sub}}^2 = \emptyset$
- Each subsequence must sample at least a quarter of the samples with each quarter of $\mathbf{S}_{\text{up}}$:

$$\left| \mathcal{X}_{\text{sub}}^i \cap \{(j-1)\frac{T_{\text{up}}}{4}, \dots, j\frac{T_{\text{up}}}{4}\} \right| \geq \left\lfloor \frac{T_{\text{int}}}{4} \right\rfloor, \quad j \in \{1, \dots, 4\} \quad (2)$$

Step 3: We resample the sequences to get the original temporal characteristics, as shown in Figure 5d.

This is done in two steps:

1. The time indices of each subsequence are stretched to cover the full range $[1, T]$ while keeping their relative positions.
2. The values are then interpolated to align with the original time steps in $\mathcal{X}$.

¹https://github.com/antoinesaget/ts_ssl

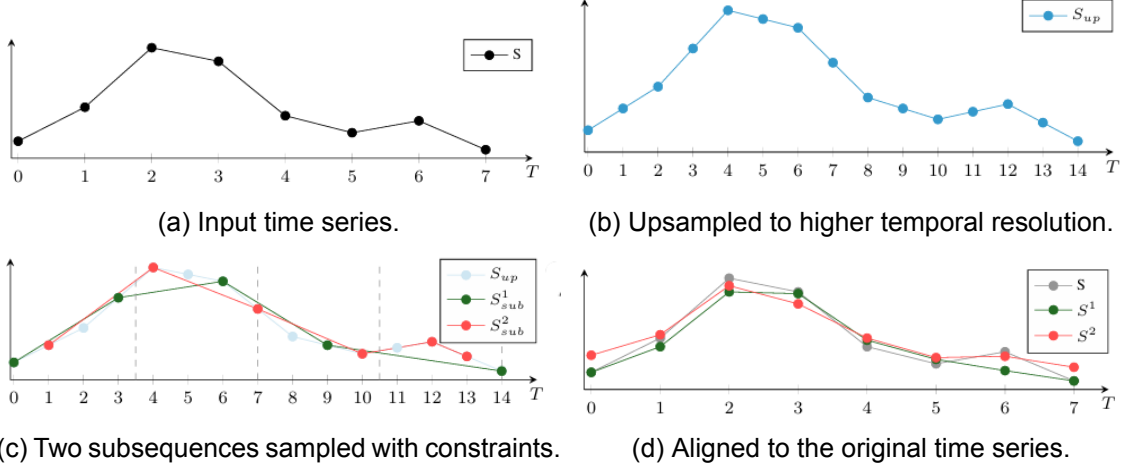


Figure 5: Resampling augmentation steps, adapted from our team’s GitHub¹ repository.

3.1.2 Window Slice

Window Slice is an augmentation method, it extracts a temporal subwindow from the original series and then interpolates back to the original length (Guennech, Malinowski, & Tavenard, 2016). This technique is suitable for long sequences, where every local segment may contain sufficient semantic information. It chooses a part of the time series randomly and interpolates for getting the original temporal length, while it preserves spatial and spectral dimensions. By doing this, it provides a new view of the same event, simulating noisy observations.

Application to Satellite Data. We are working in multispectral satellite time series, each sample is represented as a 3D tensor of shape (G, T, C) , where G represents number of pixels or spatio-temporal samples, T denotes the temporal length and C is number of bands. This approach extracts a temporal window (e.g. 70% of T) from each sample and interpolates back to T time steps for each spectral feature independently.

Mathematical Formulation. Given a time series $x \in \mathbb{R}^{G \times T \times C}$, we extracted a temporal segment for each sample x_i :

$$x_i^{\text{slice}} = x_i[t_s : t_s + L], \quad (3)$$

where t_s is random start point. This segment is resampled to T time steps:

$$x_i^{ws}[:, c] = \text{interp}(\text{linspace}(0, L - 1, T), x_i^{\text{slice}}[:, c]), \quad \forall c \in [1, C]. \quad (4)$$

An application of the Window Slice transformation is illustrated in Figure 6, where 70% of the temporal window is extracted.

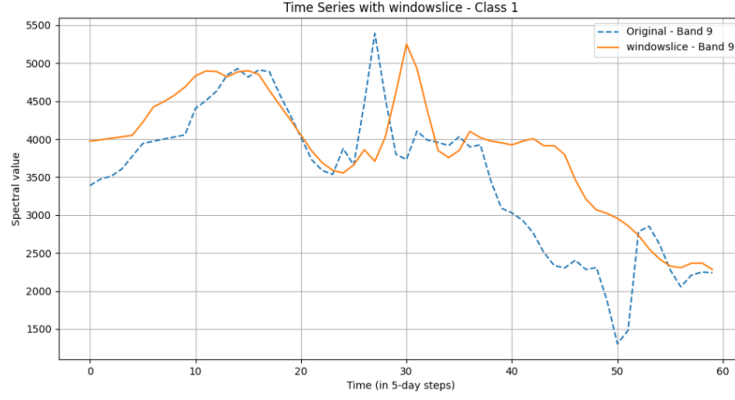


Figure 6: Window Slice augmentation example.

It modifies the content, but it doesn't affect the temporal dimension. So this approach can be used to simulate real-world situations, such as cloud cover, incomplete crop observations, etc. It is also useful in case of missing observations, which makes the model robust.

3.1.3 Time Masking

The *Time Masking* method applies the principle of Masked Signal Modeling (MSM) to time series data (Liu et al., 2024). This transformation masks a portion of the temporal dimension, encouraging the model to learn meaningful representations by reconstructing missing temporal patterns.

The steps of this transformation are as follows.

- The first step is determining the number of time steps to be masked in every sample using `mask_ratio`.
- The second step includes randomly selecting time indices to be masked.
- Then features of the selected time steps are zeroed.

Application to Satellite Data. This approach is based on a multispectral series where each sample is arranged as a tensor (G,T,C) as described in Section 3.1.2. For every sample, we randomly select $M = \text{mask_ratio} \cdot T$ time steps. These time steps are zeroed for all channels.

Mathematical Formulation. The masking operation is applied by element-wise multiplication with a binary mask tensor:

$$\text{masked_data} = x \cdot \text{mask}, \quad (5)$$

where $x \in \mathbb{R}^{G \times T \times C}$ is the input time series, and mask is zero for selected time steps and one otherwise. This helps to simulate temporal dropouts caused by clouds, sensor failures, or data disruptions, which helps the model in learning stronger temporal relationships.

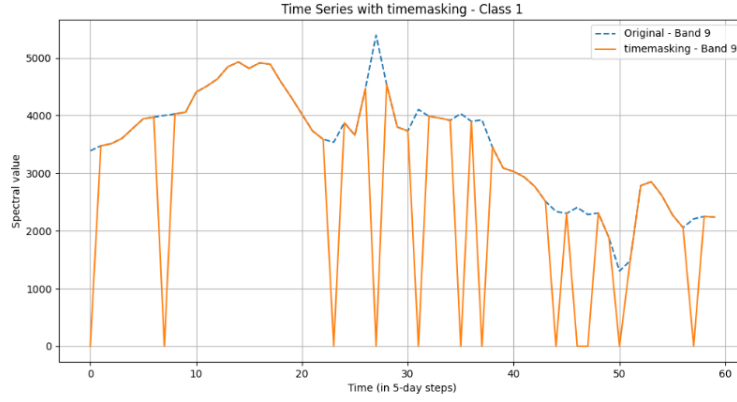


Figure 7: Time Masking augmentation example.

An application of the Time Masking transformation is illustrated in Figure 7, where 20% of the data points are masked. The masked sections are highlighted in orange, while the blue ones show the original sections.

3.1.4 MixUp

MixUp is a data augmentation method, which generates new samples by mixing parts of time series (Aggarwal & Srivastava, 2023). This method brings smoother transitions in the future space and makes the models more generalizable.

MixUp has the following key steps:

- The first step is sampling λ (mixing coefficient) from a Beta distribution with α positive parameter, which determines the mixing strength.
- New combinations of time series samples are created by shuffling the batch.
- Weights λ and $1 - \lambda$ are used to generate combinations of the real and shuffled samples.

Application to Satellite Data. This approach is based on a multispectral series where each sample is arranged as a tensor (G,T,C) as described in Section 3.1.2. MixUp shuffles data in batch dimension. Using a combination of original and shuffled samples, it provides new variations throughout the entire time series. It not only makes the model learn smoother decisions but also improves its resistance to the natural variability of satellite data.

Mathematical Formulation. MixUp transformation can be described as:

$$x_{\text{mixed}} = \lambda x + (1 - \lambda)x_{\text{shuffled}}, \quad (6)$$

where x is the original time series batch, x_{shuffled} is the batch after shuffling.

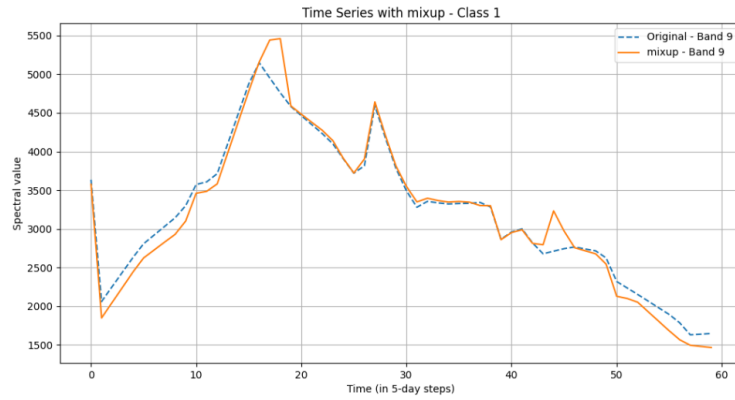


Figure 8: MixUp augmentation example.

Application of the MixUp method is shown in Figure 8, Each new sample is the combination of original and randomly selected sample.

3.1.5 Jittering

Jittering is one of the popular augmentation techniques, it adds random Gaussian noise to the time series (Iglesias, Talavera, González-Prieto, Mozo, & Gómez-Canaval, 2023; Iwana & Uchida, 2021).

It can simulate real-world noisy data. Using inherent noise, it generates new samples. The mean and standard deviation of Gaussian distribution establish the form of the distortion, which makes this technique generalizable to different needs.

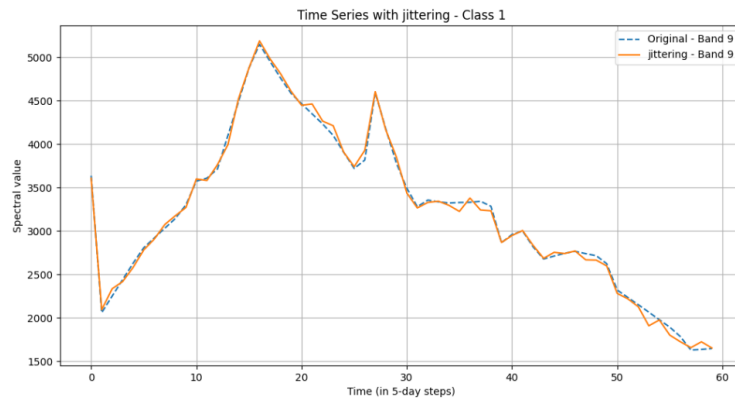


Figure 9: Jittering augmentation example.

An application of the Jittering is illustrated in Figure 9, It simulates real-world noisy data, at the same time preserves its structure.

3.1.6 Resizing

The resizing data augmentation method modifies the temporal resolution of the data that random crops the time series and rescales it back to the original size (Liu et al., 2024). This methodology involves temporal distortion which forces the model to be invariant to timing variations in the signal. The degree of this change is controlled by a parameter, *magnitude*. A crop length is sampled randomly from a uniform distribution in $[1 - \text{magnitude}, 1]$, which means that smaller values preserve more of the sequence. Then the cropped segment is interpolated back to the original sequence length, preserving the time series dimensions.

Mathematical Formulation. Given a time series $x \in \mathbb{R}^{G \times T \times C}$, for each sample x_i , a random resize factor r_i is sampled from the interval $[1 - \alpha, 1]$. A cropped segment x_i^{crop} is taken from a random starting point, and then interpolated back to T steps:

$$x_i^{\text{resized}} = \text{Interpolate}(x_i[t : t + \lfloor r_i T \rfloor]) \quad (7)$$

where t is a random starting index that the crop fits within the time range, and linear interpolation is performed through the temporal axis.

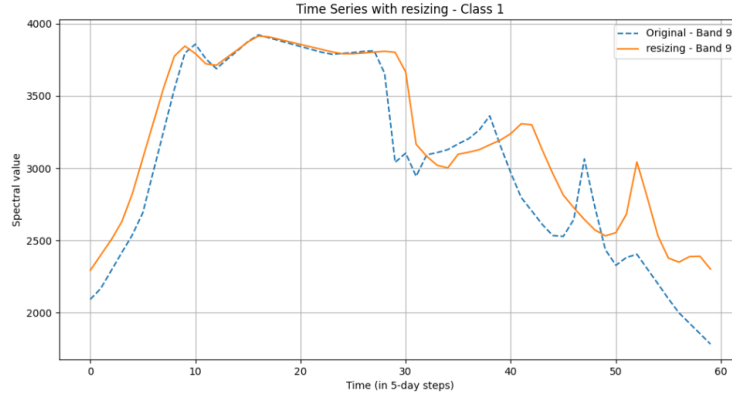


Figure 10: Resizing augmentation example.

Figure 10 illustrates an application of the Resizing augmentation. A random part of the time series is cropped and then interpolated to get the original length. This approach allows the models to learn from temporal distortions, which preserves the overall shape of the signal. It can be used when the model is not robust for time warping.

3.1.7 Window Warping

Window Warping is a temporal distortion data augmentation method, which applies local time distortions, it compresses or stretches a sub-segment of a time series (Guenneec et al., 2016). This method simulates variations of observation speed, but it still keeps the structure of the signal.

The method randomly chooses a window with a predefined length. Then it is time to warp the window using a randomly chosen scaling factor from a predefined set, which either compresses

or expands the segment. The modified segment is then inserted back into the sequence, and the whole time series is linearly interpolated for the original temporal resolution.

Application to Satellite Data. This approach is based on a multispectral sequence where each sample is arranged as a tensor (G,T,C) as described in Section 3.1.2. Window Warping is implemented independently to each sample, it warps randomly selected temporal window and then interpolates the whole sequence back to length T . This technique simulates localized variations of temporal progression, for example differing crop development speeds across regions or seasons, without changing its global structure.

Mathematical Formulation. Given a time series $x \in \mathbb{R}^{G \times T \times C}$, a window of length $L = \lfloor \lambda \cdot T \rfloor$ is chosen and warped by a random scaling factor α_i :

$$\text{segment}_{\text{warped}} = \text{interp}(\text{linspace}(0, L - 1, \alpha_i \cdot L), x_i[t_s : t_e, c]), \quad (8)$$

The warped segment is then reconstructed and resampled back to length T with:

$$x_i^{\text{ww}}[:, c] = \text{interp}(\text{linspace}(0, T' - 1, T), x_i^{\text{concat}}[:, c]) \quad (9)$$

where T' is the intermediate temporal length after the window warping.

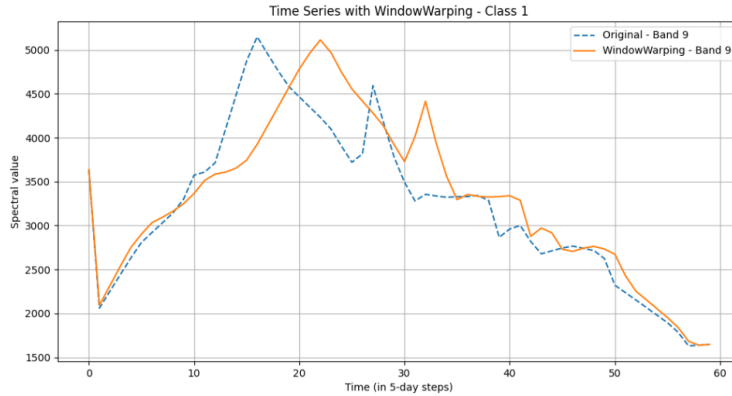


Figure 11: Window Warping augmentation example.

Figure 11 shows an example of the Window Warping transformation to Satellite-2 time series data, where a window from the time series is warped and the entire sequence is rescaled to the original length, introducing controlled temporal distortions.

3.1.8 Scaling

Scaling is a amplitude-based data augmentation technique that multiplies the input signal by a random factor close to 1 (Iglesias et al., 2023). This method aims to simulate amplitude changes or reflectance intensity, which very commonly occurs in satellite data due to atmospheric conditions.

A separate scaling factor per sample in a batch is sampled from the uniform distribution $[1 - \gamma, 1 + \gamma]$, where γ is maximum deviation from the identity transformation. This factor is then implemented

to the entire sample, It alters the scale but keeps its shape and structure.

Application to Satellite Data. This approach is based on a multispectral sequence where each sample is arranged as a tensor (G,T,C) as described in Section 3.1.2. We apply Scaling independently per sample through all temporal steps and spectral channels. This technique implements realistic physical variations without disrupting the internal temporal or spectral correlations.

Mathematical Formulation. Given a time series $x \in \mathbb{R}^{G \times T \times C}$, s is random scaling factor. The result is calculated as follows:

$$x^{\text{scaled}} = s \cdot x \quad (10)$$

Figure 12 shows application of Scaling transformation to Satellite-2 time series data.

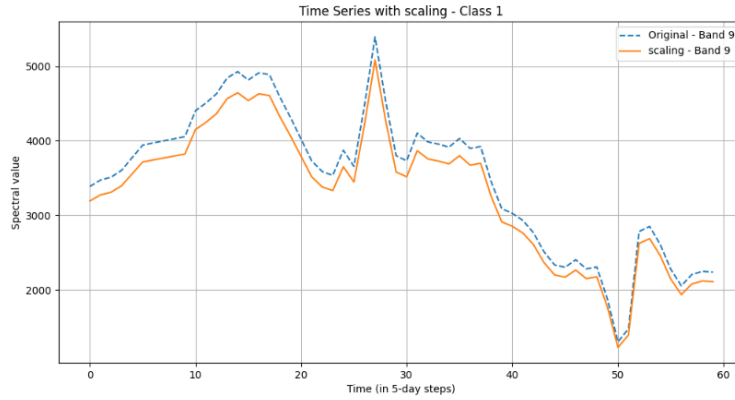


Figure 12: Scaling augmentation example.

This augmentation method implements a global proportional shift in signal intensity. It helps the model ignore the amplitude variations and sensor differences, but it still maintains the relative dynamics between time and spectral channels.

3.1.9 Temporal Dropout

Temporal Dropout is a temporal data augmentation technique which drops out part of the temporal observations from a series (Yuan, Lin, Xin, Zhou, & Liu, 2025), then interpolating the dropped sequence back to a sequence of original length. This approach is especially useful in multispectral time series like satellite data, which is contaminated by clouds, etc.

Unlike previous studies, this technique helps model for learning more robust representation against observation interruptions.

Application to Satellite Data. This approach is based on a multispectral sequence where each sample is arranged as a tensor (G,T,C) as described in Section 3.1.2. A temporal subsequence of variable length is kept (e.g., 70% of the time steps), and the rest is removed. Then missing values are then linearly interpolated over time for each band independently, and combined, as a result we get a reconstruction with the same length as input series.

Mathematical Formulation. Let $x \in \mathbb{R}^{G \times T \times C}$ denote a time series. For each series x_i , we randomly sample a subset of temporal indices S , with cardinality:

$$|S| = \lfloor (1 - r) \cdot T \rfloor, \quad (11)$$

where r is the dropout rate. The complete series is then reconstructed by interpolation at indices S :

$$x_i^{td}[:, c] = \text{interp}(S, x_i[S, c], \{0, \dots, T - 1\}). \quad (12)$$

This equation simulates the loss of temporal observations in time series, while preserving the global structure.

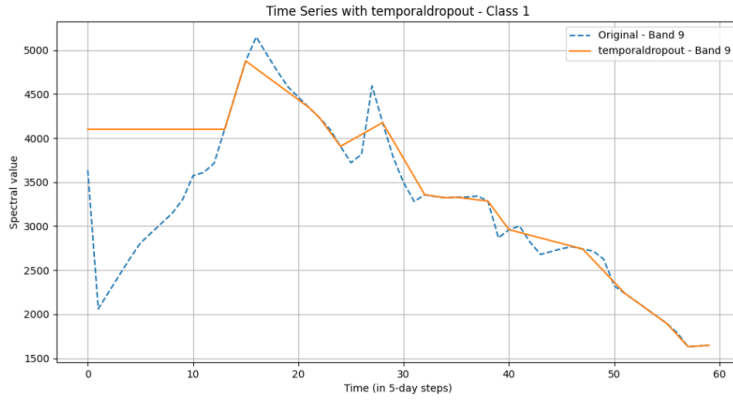


Figure 13: Temporal Dropout augmentation example.

Application of the Temporal Dropout method is shown in Figure 13, it drops out random parts of time series and reconstructs with interpolation. This augmentation method makes models to handle real-world data interruptions.

3.1.10 StiefelGen

StiefelGen is a geometric data augmentation technique, it uses the Siefel manifold to produce structured perturbations over time series (Cheema & Sugiyama, 2024). It doesn't directly modify raw data, instead it works in specific geometric space using isometries.

Mathematical Formulation. A time series $X \in \mathbb{R}^{T \times d}$ is decomposed by SVD: $X = USV^T$, where U and V are orthonormal, and S has singular values. The perturbations Δ_U, Δ_V are implemented in tangent spaces:

$$U' = \exp_U(\Delta_U), \quad V' = \exp_V(\Delta_V), \quad X' = U'SV'^T \quad (13)$$

Given $\Delta = Q\Sigma$ (from skew-symmetric decomposition), the exponential:

$$\exp_U(\Delta) = U \cos(\Sigma) + Q \sin(\Sigma) \quad (14)$$

Application to Satellite Data. Data $\mathbf{x} \in \mathbb{R}^{G \times T \times C}$ is reshaped to $X \in \mathbb{R}^{T \times (G \cdot C)}$, then it is perturbed and reshaped again. Only directions u_i, v_i are modified for realistic, structure-preserving variations. Singular values are unchanged.

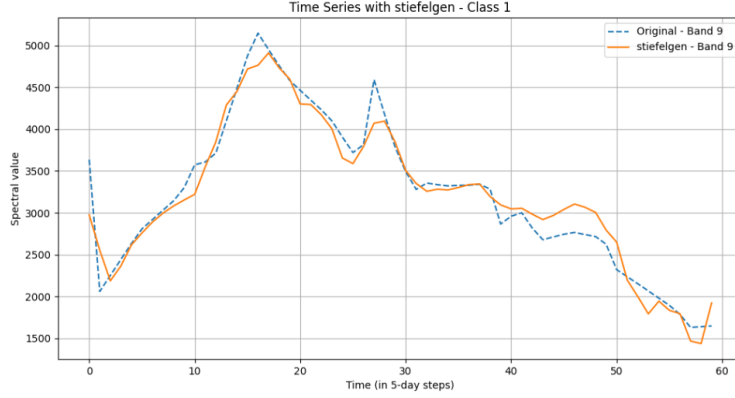


Figure 14: StiefelGen augmentation example.

Application of StiefelGen data augmentation technique to time series is shown in Figure 14, where structured perturbations are added. It preserves overall amplitude and base structure of the data, while creating realistic variations.

3.2 Experimental Setup

3.2.1 Dataset

The **FranceCrops**² dataset is a rich satellite resource designed for the classification of agricultural plots in France. It is based on multispectral observations from the Sentinel-2 satellite (L2A product, atmospherically corrected), collected between February and November 2022. Every sample is a 3D tensor that contains the temporal evolution of multiple spectral elements of the same parcel.

Sample Structure:

$$x \in \mathbb{R}^{100 \times 60 \times 12}$$

- 100 pixels extracted from each agricultural plot.
- 60 time steps, sampled every 5 days.
- 12 Sentinel-2 L2A bands, described in Figure 15.

Data Splits:

According to huggingface² source:

²<https://huggingface.co/datasets/saget-antoine/francecrops>

- **Contrastive:** 1 million unlabeled samples for SSL.

- **Supervised:**

Training: 2,000 samples (20 classes, 100 samples per class).

Validation: 2,000 samples (20 classes, 100 samples per class).

Test: 20,000 samples (20 classes, 1,000 samples per class).

Figure 15 illustrates the spectral time series of blé tendre d’hiver (soft winter wheat) from the FranceCrops² dataset (1 February-30 November).

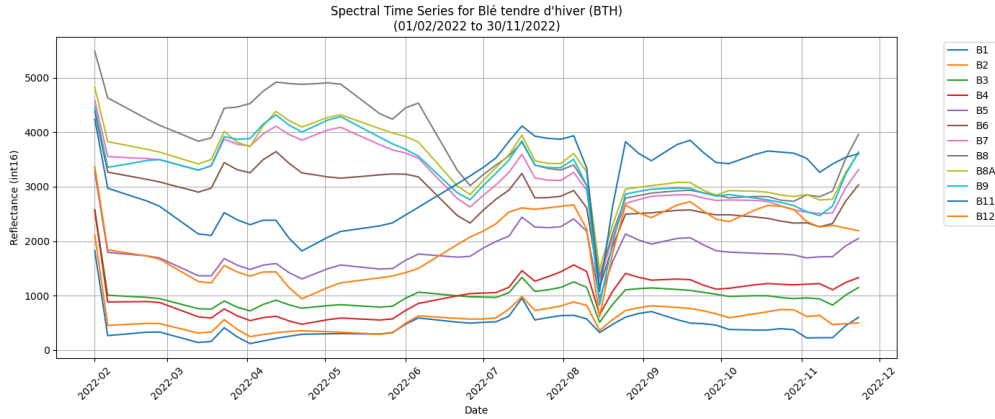


Figure 15: Visualization of the spectral time series of blé tendre d’hiver (soft winter wheat).

3.2.2 Architecture

As explained before, we compare SSL frameworks on the FranceCrops dataset. The encoder is ResNet adapted to time series (Z. Wang, Yan, & Oates, 2016), also a projection head (X. Chen & He, 2021).

We apply ResNet to inputs with 12 channels and 60 time steps. The process consists of residual blocks that include multiple convolutional layers, ReLU activation, and a residual connection (between input and output). Once the features are extracted, we apply global temporal pooling (AdaptiveAvgPool1d) to get an embedded vector with 512 dimensions (Saget, Lafabregue, Cornuéjols, & Gançarski, 2024).

Each SSL framework has its own projection head for mapping the embeddings into a latent space:

BYOL model has a projection head (dimension: 512 to 256), MLP prediction head (dimension: 256 to 256), and momentum encoder with momentum of 0.996 (Grill et al., 2020).

In **SimCLR**, the projection head needs NT-Xent contrastive loss (temperature 0.1) according to T. Chen, Kornblith, Norouzi, and Hinton (2020).

VICReg architecture includes a 2-layer MLP projection head (dimension: 512 to 128) with BatchNorm and between layers it has ReLU (Bardes et al., 2022).

In **MoCo**, there is a projection head (dimension: 512 to 128), a momentum encoder is updated with 0.999, and a queue (dynamic) for storing negative samples, but there is no predictor (He et al., 2020).

The Augmentation Strategies are: **Copy** generates views from the same input sample, while **Group** samples the views from different inputs in the same group.

In this experiment, we used the **Copy** strategy, which better suits the SSL characteristics.

3.2.3 Training and Evaluation Settings

This section is adapted from GitHub repository¹.

For training, **Stochastic Gradient Descent (SGD)** optimizer is used, with the hyperparameters: momentum is 0.9, learning rate: $2e-3$ to $5e-2$ (first 20% of training), then $5e-5$, batch size is 1024, and step size is 50,000.

After training was completed, we **evaluated** the encoder by **linear evaluation**, with the hyperparameters: max iterations are 2000, tolerance is $1e-5$, and regularization parameter (C) is 1.0.

We assess model accuracy using 5, 10, 20, 50, and 100 samples per class. The final results are acquired by the **majority accuracy**.

4 Experimental Results

Tables 1-4 compare the performance of 10 individual transformations and 6 combinations of well-performed transformations among them, across frameworks: SimCLR, VICReg, MoCo, and BYOL. The results show that combining augmentations, especially combinations of Temporal Dropout, Resampling, and Window Slice achieve better performance across the training samples (5, 10, 20, 50, 100) per class.

SimCLR. Table 1 shows the results for SimCLR. The Resampling + Window Slice combination has the highest accuracy and it is better than its individual components. Other combinations, like Temporal Dropout + Window Slice and Resampling + Temporal Dropout, also show strong performance. Some individual methods are not useful for this framework, like Scaling.

Table 1: Performance comparison of augmentation techniques in SimCLR.

Augmentation Strategy	Training Samples per Class				
	5	10	20	50	100
Scaling	20	24	29	35	38
Stiefelgen	33	44	53	59	63
Window Warping	41	49	55	61	64
Jittering	43	50	57	62	66
MixUp	42	50	59	66	70
Resizing	47	54	60	67	70
Window Slice	54	62	70	76	80
Resampling	60	67	71	76	78
Time Masking	56	65	73	79	82
Time Masking + Window Slice	59	67	74	80	83
Temporal Dropout	61	68	75	81	84
Time Masking + Temporal Dropout	63	70	75	81	84
Resampling + Time Masking	63	70	76	81	84
Temporal Dropout + Window Slice	65	72	77	82	84
Resampling + Temporal Dropout	66	72	77	81	84
Resampling + Window Slice	68	73	78	81	84

VICReg. Table 2 shows the results for VICReg. Similar to SimCLR, Resampling + Window Slice and Temporal Dropout + Window Slice achieved high accuracies. Interestingly, the Resampling method individually is among the highest performances, which shows that VICReg is taking more advantage of disturbing the temporal structure of data.

Table 2: Performance comparison of augmentation techniques in VICReg.

Augmentation Strategy	Training Samples per Class				
	5	10	20	50	100
MixUp	20	23	28	35	40
Scaling	37	43	48	53	56
Stiefelgen	43	49	55	61	65
Window Warping	47	53	59	64	67
Jittering	45	52	58	64	68
Resizing	56	62	67	71	74
Time Masking + Window Slice	57	65	72	78	81
Window Slice	58	66	72	77	80
Time Masking	59	67	74	80	82
Time Masking + Temporal Dropout	62	68	74	80	82
Temporal Dropout	62	69	75	80	83
Resampling + Time Masking	62	69	75	81	83
Resampling + Temporal Dropout	64	70	75	80	83
Resampling	68	73	77	81	83
Temporal Dropout + Window Slice	66	72	77	82	84
Resampling + Window Slice	68	73	77	81	84

MoCo. Table 3 shows the results for MoCo. Resampling + Window Slice achieves the highest accuracy. But, some combinations did not show performance as we expected. This suggests that despite Time Masking being effective individually, its combinations may not be effective in MoCo.

Table 3: Performance comparison of augmentation techniques in MoCo.

Augmentation Strategy	Training Samples per Class				
	5	10	20	50	100
Scaling	24	32	40	48	52
Window Warping	45	53	61	69	72
Jittering	41	48	55	62	65
Resizing	44	52	60	67	71
Time Masking + Temporal Dropout	42	52	61	69	71
Stiefelgen	50	58	64	70	73
Time Masking + Window Slice	40	51	62	72	77
MixUp	51	58	64	70	75
Window Slice	57	65	72	78	81
Time Masking	55	63	71	78	82
Temporal Dropout	59	67	74	80	83
Resampling + Time Masking	58	67	74	80	84
Resampling + Temporal Dropout	60	67	73	79	82
Temporal Dropout + Window Slice	59	68	75	81	83
Resampling	65	71	76	80	83
Resampling + Window Slice	64	71	77	81	84

BYOL. Table 4 presents the results for BYOL. The results show that Resampling + Window Slice stands at the top. As we expected, the combinations, that disturb temporal structure perform better in the BYOL framework.

Table 4: Performance comparison of augmentation techniques in BYOL.

Augmentation Strategy	Training Samples per Class				
	5	10	20	50	100
Jittering	39	45	50	55	59
Stiefelgen	42	50	56	63	66
Scaling	44	52	59	65	68
Resizing	53	60	65	70	72
Window Slice	55	60	65	69	72
Window Warping	52	60	66	71	74
MixUp	53	61	68	74	77
Time Masking	60	68	74	79	82
Time Masking + Window Slice	61	69	74	80	83
Resampling + Time Masking	60	68	75	81	84
Time Masking + Temporal Dropout	62	69	75	80	83
Resampling + Temporal Dropout	64	70	75	80	82
Temporal Dropout	65	71	76	81	83
Temporal Dropout + Window Slice	66	72	77	81	83
Resampling	72	76	79	81	83
Resampling + Window Slice	70	76	79	82	84

5 Discussion

This section presents the findings of this work. And we also talk about the limitations.

5.1 Findings

Benefits of Combined Augmentations: The results support our hypothesis that using combined augmentation techniques-especially Temporal Dropout, Resampling, and Window Slice reduces label dependence. When we look at the results, we can say that these combinations almost worked well with just 20-50 samples per class, while for some results individual ones use 100 samples. This means that label dependence was reduced approximately 2-5 times.

Framework-Specific Performance: BYOL and VICReg performed well with combined augmentations. This supports our hypothesis. SimCLR also performed better, but not as consistently as BYOL and VICReg. It may be caused by depending on negative sampling. The least improvements were observed in MoCo. Some combinations were performed well, but some combinations with Time Masking and Temporal Dropout didn't provide consistent results. One possible reason is MoCo depends on negative samples, which can make it challenging.

Role of Temporal Augmentations: The results show that methods like Temporal Dropout, Resampling, and Window Slice, which alter the temporal structure of the data are more effective.

5.2 Limitations

Long Training Time and GPU Usage: One single training takes 1-2 days and at least 13 GB of GPU memory, making the training process challenging.

Limited GPU Resources: As training takes at least 13 GB of GPU memory, only 2 GPUs were suitable. Other lab members also use these GPUs. And sometimes some GPUs were not available because of system updates and problems.

Hyperparameter Tuning: Each augmentation has specific hyperparameters that need to be tested for reliable evaluation. It increases training runs.

6 Conclusions

In this study, we investigated the performance of 10 individual transformations and 6 combinations of well-performed transformations among them on the Satellite Image Time Series(SITS) using FranceCrops dataset, across frameworks: SimCLR, VICReg, MoCo, and BYOL.

The results show that combining augmentations, especially Resampling, Temporal Dropout and Window Slice perform better than using a single method. They reduce label dependence approximately 2-5 times. The combination Resampling + Window Slice is the best-performed combination among all the methods we implemented. The frameworks VICReg and BYOL benefited the most

improvements from this strategy, SimCLR also showed improvements, but MoCo showed less satisfied results in general.

These findings also suggest that the augmentations that alter the temporal structure of the data are effective for remote sensing time series data. However, this study has limitations, such as long training duration and high GPU memory requirements, other challenges include hyperparameter tuning.

For future work, we plan to apply this strategy to other types of time series data, not only remote sensing. We also want to implement the best-performing augmentations randomly at the batch level, so each batch is modified by a random augmentation or combination. And, we are interested to explore generative-based augmentations for remote sensing.

References

- Aggarwal, K., & Srivastava, J. (2023). Embarrassingly simple mixup for time-series. *CoRR*, *abs/2304.04271*. doi: 10.48550/arxiv.2304.04271
- Bardes, A., Ponce, J., & Lecun, Y. (2022, April). VICReg: Variance-Invariance-Covariance Regularization For Self-Supervised Learning. In *ICLR 2022 - International Conference on Learning Representations*. Online, United States. Retrieved from <https://inria.hal.science/hal-03541297>
- Cheema, P., & Sugiyama, M. (2024). Stiefelgen: A simple, model agnostic approach for time series data augmentation over riemannian manifolds. *CoRR*, *abs/2402.19287*. doi: 10.48550/arXiv.2402.19287
- Chen, T., Kornblith, S., Norouzi, M., & Hinton, G. (2020). A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th international conference on machine learning*. JMLR.org.
- Chen, T., Kornblith, S., Swersky, K., Norouzi, M., & Hinton, G. (2020). Big self-supervised models are strong semi-supervised learners. In *Proceedings of the 34th international conference on neural information processing systems*. Red Hook, NY, USA: Curran Associates Inc.
- Chen, X., & He, K. (2021, June). Exploring simple siamese representation learning. In *IEEE/CVF conference on computer vision and pattern recognition (cvpr)* (p. 15745-15753). doi: 10.1109/CVPR46437.2021.01549
- Cong, Y., Khanna, S., Meng, C., Liu, P., Rozi, E., He, Y., ... Ermon, S. (2022). Sat-mae: Pre-training transformers for temporal and multi-spectral satellite imagery. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, & A. Oh (Eds.), *Advances in neural information processing systems* (Vol. 35, pp. 197–211). Curran Associates, Inc. Retrieved from https://proceedings.neurips.cc/paper_files/paper/2022/file/01c561df365429f33fcd7a7faa44c985-Paper-Conference.pdf
- Drusch, M., Del Bello, U., Carlier, S., Colin, O., Fernandez, V., Gascon, F., ... Bargellini, P. (2012). Sentinel-2: Esa's optical high-resolution mission for gmes operational services. *Remote Sensing of Environment*, 120, 25-36. (The Sentinel Missions - New Opportunities for Science) doi: 10.1016/j.rse.2011.11.026
- Gascon, F., Bouzinac, C., Thépaut, O., Jung, M., Francesconi, B., Louis, J., ... Fernandez, V. (2017). Copernicus sentinel-2a calibration and products validation status. *Remote Sensing*, 9(6). doi: 10.3390/rs9060584
- Grill, J.-B., Strub, F., Altché, F., Tallec, C., Richemond, P. H., Buchatskaya, E., ... Valko, M. (2020). Bootstrap your own latent a new approach to self-supervised learning. In *Proceedings of the 34th international conference on neural information processing systems*. Red Hook, NY, USA: Curran Associates Inc.
- Guenec, A. L., Malinowski, S., & Tavenard, R. (2016). Data augmentation for time series classification using convolutional neural networks.. Retrieved from <https://api.semanticscholar.org/CorpusID:3907864>
- Guo, X., Lao, J., Dang, B., Zhang, Y., Yu, L., Ru, L., ... Li, Y. (2024, June). Skysense: A multi-modal remote sensing foundation model towards universal interpretation for earth observa-

- tion imagery. In *IEEE/CVF conference on computer vision and pattern recognition (cvpr)* (p. 27662-27673). doi: 10.1109/CVPR52733.2024.02613
- He, K., Chen, X., Xie, S., Li, Y., Dollár, P., & Girshick, R. (2022, June). Masked autoencoders are scalable vision learners. In *IEEE/CVF conference on computer vision and pattern recognition (cvpr)* (p. 15979-15988). doi: 10.1109/CVPR52688.2022.01553
- He, K., Fan, H., Wu, Y., Xie, S., & Girshick, R. (2020). Momentum contrast for unsupervised visual representation learning. In *IEEE/CVF conference on computer vision and pattern recognition (cvpr)* (p. 9726-9735). doi: 10.1109/CVPR42600.2020.00975
- Hénaff, O. J., Srinivas, A., De Fauw, J., Razavi, A., Doersch, C., Eslami, S. M. A., & Van Den Oord, A. (2020). Data-efficient image recognition with contrastive predictive coding. In *Proceedings of the 37th international conference on machine learning*. JMLR.org.
- Iglesias, G., Talavera, E., González-Prieto, A., Mozo, A., & Gómez-Canaval, S. (2023, 03). Data augmentation techniques in time series domain: a survey and taxonomy. *Neural Computing and Applications*, 35(14), 10123–10145. doi: 10.1007/s00521-023-08459-3
- Iwana, B. K., & Uchida, S. (2021, 07). An empirical survey of data augmentation for time series classification with neural networks. *PLOS ONE*, 16(7), 1-32. doi: 10.1371/journal.pone.0254841
- Jakubik, J., Roy, S., Phillips, C., Fraccaro, P., Godwin, D., Zadrozny, B., ... Ramachandran, R. (2023, 11). *Foundation models for generalist geospatial artificial intelligence*. doi: 10.48550/arXiv.2310.18660
- Jean, N., Wang, S., Samar, A., Azzari, G., Lobell, D., & Ermon, S. (2019). Tile2vec: unsupervised representation learning for spatially distributed data. In *Proceedings of the thirty-third aaai conference on artificial intelligence and thirty-first innovative applications of artificial intelligence conference and ninth aaai symposium on educational advances in artificial intelligence*. AAAI Press. doi: 10.1609/aaai.v33i01.33013967
- Liu, Z., Alavi, A., Li, M., & Zhang, X. (2024, 07). *Guidelines for augmentation selection in contrastive learning for time series classification*. doi: 10.48550/arXiv.2407.09336
- Mañas, O., Lacoste, A., Giró-i Nieto, X., Vazquez, D., & Rodríguez, P. (2021). Seasonal contrast: Unsupervised pre-training from uncurated remote sensing data. In *IEEE/CVF international conference on computer vision (iccv)* (p. 9394-9403). doi: 10.1109/ICCV48922.2021.00928
- Reed, C. J., Gupta, R., Li, S., Brockman, S., Funk, C., Clipp, B., ... Darrell, T. (2023, 10). Scale-mae: A scale-aware masked autoencoder for multiscale geospatial representation learning. In *Proceedings of the IEEE/CVF international conference on computer vision (iccv)* (pp. 4065–4076). doi: 10.1109/ICCV51070.2023.00378
- Saget, A., Lafabregue, B., Cornuéjols, A., & Gançarski, P. (2024). Learning from few labeled time series with segment-based self-supervised learning: Application to remote-sensing. In *Proceedings of spaice 2024: The first joint european space agency/iaa conference on ai in and for space* (pp. 275–279).
- Tseng, G., Zvonkov, I., Purohit, M., Rolnick, D., & Kerner, H. (2023, 04). Lightweight, pre-trained transformers for remote sensing time series. *arXiv preprint arXiv:2304.14065*. doi: 10.48550/arXiv.2304.14065
- Wang, Y., Braham, N. A. A., Xiong, Z., Liu, C., Albrecht, C. M., & Zhu, X. X. (2023, Sep.). Ssl4eo-s12: A large-scale multimodal, multitemporal dataset for self-supervised learning in earth

observation [software and data sets]. *IEEE Geoscience and Remote Sensing Magazine*, 11(3), 98-106. doi: 10.1109/MGRS.2023.3281651

Wang, Z., Yan, W., & Oates, T. (2016). Time series classification from scratch with deep neural networks: A strong baseline. *2017 International Joint Conference on Neural Networks (IJCNN)*, 1578-1585.

Yuan, Y., Lin, L., Xin, Q., Zhou, Z.-G., & Liu, Q. (2025). An empirical study on data augmentation for pixelwise satellite image time-series classification and cross-year adaptation. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 18, 5172-5188. doi: 10.1109/JSTARS.2025.3527017